รายวิชา IT368

Cloud Computing

Lecture 6

Container and Docker Fundamentals

พื้นฐานคอนเทนเนอร์และ Docker

เข้าใจแนวคิด เรียนรู้การใช้งาน และก้าวสู่การพัฒนาแอปพลิเคชันบนคอนเทนเนอร์



Build Knowledge | Hands-on Practice | Real-world Skills | Prepare for the Cloud

ปัญหาของการติดตั้งแอปพลิเคชันแบบดั้งเดิม

การติดตั้งแอปพลิเคชันแบบทั่วไป ต้องเตรียมสภาพแวดล้อม (environment) ให้เหมือนกันในทุกเครื่อง ซึ่งมักเกิดปัญหาเมื่อนำแอปพลิเคชันจากเครื่องพัฒนา (developer) ไปยังเครื่องจริง (production)

เครื่องพัฒนา (Developer)



- Ubuntu 24.04
- Python 3.12
- Library A v2.0
- Library B v4.0

แอปพลิเคชัน ไม่สามารถทำงานได้

"It works on my machine!"

เครื่องจริง (Production)



- Ubuntu 22.04
- Python 3.10
- Library A v1.0
- Library B v4.0

ประเด็นสำคัญ

แอปพลิเคชันไม่ได้ต้องการแค่ซอร์สโค้ด แต่ต้องการสภาพแวดล้อมที่เหมือนกันเพื่อให้ทำงานได้อย่างถูกต้อง

ตัวอย่างปัญหา: ทำงานได้บนเครื่องพัฒนา แต่ไม่ทำงานบนเซิร์ฟเวอร์

แม้จะใช้โค้ดเดียวกัน แต่สภาพแวดล้อม (environment) ของแต่ละเครื่องอาจแตกต่างกัน
ทำให้เกิดปัญหาเมื่อย้ายแอปพลิเคชันไปยังเครื่องจริง

เครื่องพัฒนา (Developer Machine)

สภาพแวดล้อมบนเครื่องของนักพัฒนา



- Ubuntu 24.04
- Python 3.12
- Library A v2.0
- Library B v4.0

ย้ายไปใช้งาน
บนเซิร์ฟเวอร์

เครื่องจริง (Production Server)

สภาพแวดล้อมบนเซิร์ฟเวอร์จริง



- Ubuntu 22.04
- Python 3.10
- Library A v1.0
- Library B v4.0

ข้อสังเกต

แอปพลิเคชันไม่เพียงต้องการโค้ดเท่านั้น แต่ยังต้องการสภาพแวดล้อมที่เหมือนกันเพื่อให้ทำงานได้อย่างถูกต้อง

ปัญหาความขัดแย้งของไลบรารี (Dependency Conflict)

เชิร์ฟเวอร์เครื่องเดียวกันอาจต้องรันหลายแอปพลิเคชัน ซึ่งแต่ละแอปพลิเคชันอาจต้องการเวอร์ชันของไลบรารีหรือรันไทม์ที่แตกต่างกัน ทำให้เกิดความขัดแย้งในการติดตั้งและใช้งาน

ติดตั้งแอปพลิเคชันหลายตัวบนเชิร์ฟเวอร์เดียวกัน



เชิร์ฟเวอร์
(Physical/Virtual Machine)

แอปพลิเคชัน A
(Web Application)
ต้องการ Python 3.10

แอปพลิเคชัน B
(Data Processing)
ต้องการ Python 3.12

ปัญหาที่เกิดขึ้น

- ไม่สามารถติดตั้ง Python ได้ทั้ง 2 เวอร์ชันพร้อมกัน
- ไลบรารีของแต่ละแอปพลิเคชันอาจไม่เข้ากัน
- เสี่ยงต่อการกระทบกับแอปพลิเคชันอื่นที่รันอยู่บนเชิร์ฟเวอร์
- **เราจะแยกสภาพแวดล้อมของแต่ละแอปพลิเคชันอย่างไร?**

ประเด็นสำคัญ

แอปพลิเคชันแต่ละตัวควรมีสภาพแวดล้อมของตัวเอง เพื่อหลีกเลี่ยงปัญหาความขัดแย้งของไลบรารีและรันไทม์

เปรียบเทียบสถาปัตยกรรม: Virtual Machine vs Container

ทั้งสองเทคโนโลยีช่วยแยกสภาพแวดล้อมของแอปพลิเคชัน แต่มีแนวคิดและการทำงานที่แตกต่างกัน

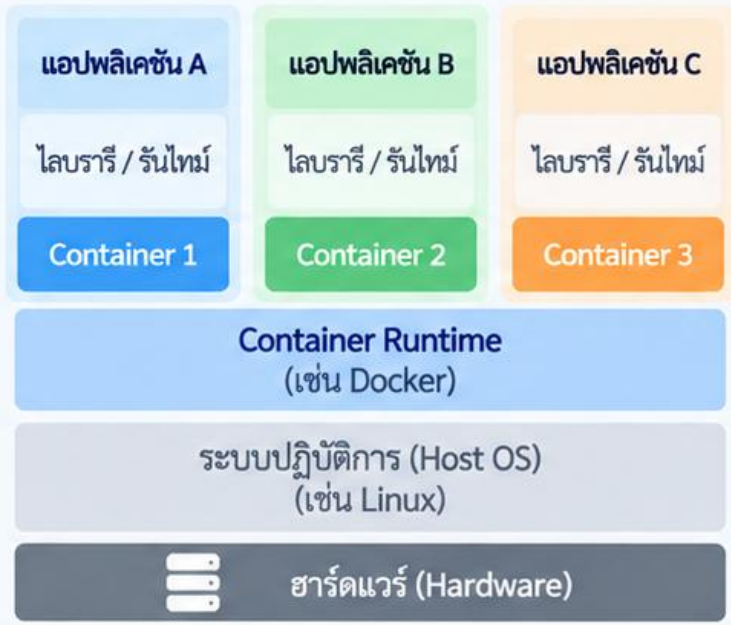
Virtual Machine (VM)

มีระบบปฏิบัติการของตัวเอง (Guest OS)



Container

ใช้ Kernel ของ Host OS ร่วมกัน



เปรียบเทียบคุณสมบัติสำคัญ

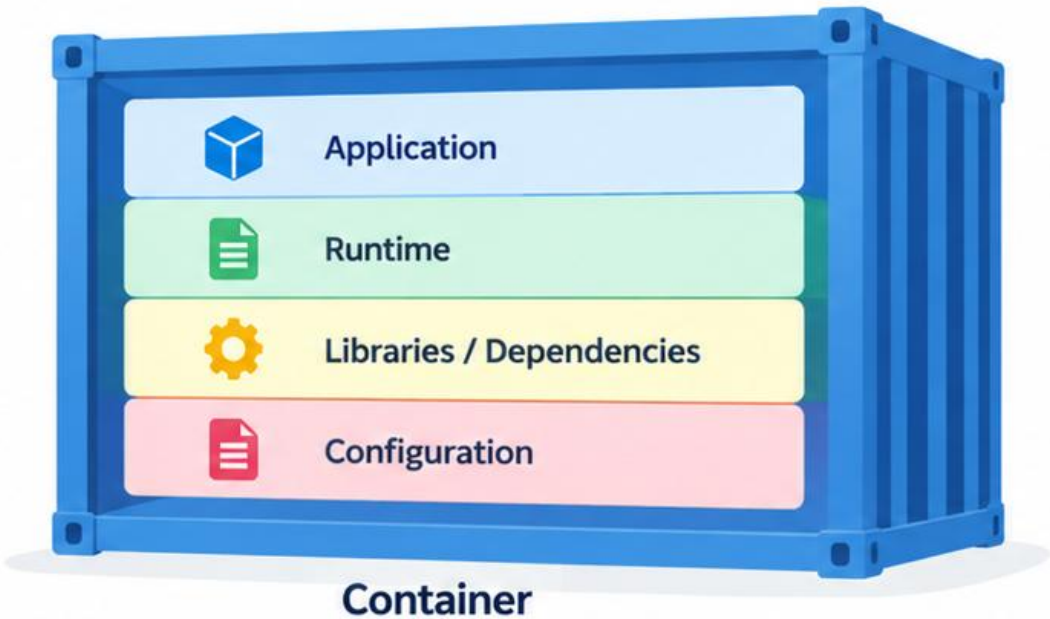
รายการ	Virtual Machine	Container
ความเป็นอิสระ	สูง (ระดับ OS)	ระดับโปรเซส
Guest OS	มี (แยกตาม VM)	ไม่มี ใช้ Kernel ร่วมกัน
ขนาด	ขนาดใหญ่ (GB)	ขนาดเล็ก (MB – GB)
เวลาเริ่มต้น	ช้า (วินาที – นาที)	เร็ว (วินาที)
ใช้ทรัพยากร	มากกว่า	น้อยกว่า
ความยืดหยุ่นในการใช้งาน	ดี	ดีมาก
ความปลอดภัย	สูงกว่า	ต่ำกว่า (ขึ้นกับการใช้งาน)

สรุป

VM และ Container ไม่ได้มาแทนที่กัน แต่เหมาะกับงานคนละประเภท ในระบบคลาวด์มักใช้ทั้งสองเทคโนโลยีร่วมกันได้

คอนเทนเนอร์ (Container) คืออะไร?

คอนเทนเนอร์คือสภาพแวดล้อมแบบแยกอิสระ (isolated environment) สำหรับรันแอปพลิเคชัน
โดยรวมทุกสิ่งที่จำเป็นต่อการทำงานของแอปพลิเคชันไว้ด้วยกัน



Container

สิ่งที่มีอยู่ในคอนเทนเนอร์

Application	โค้ดของแอปพลิเคชันที่ต้องการรัน
Runtime	เช่น Python, Node.js, Java
Libraries / Dependencies	ไลบรารีและส่วนที่แอปพลิเคชันต้องการ
Configuration	ไฟล์ตั้งค่า เช่น environment variables

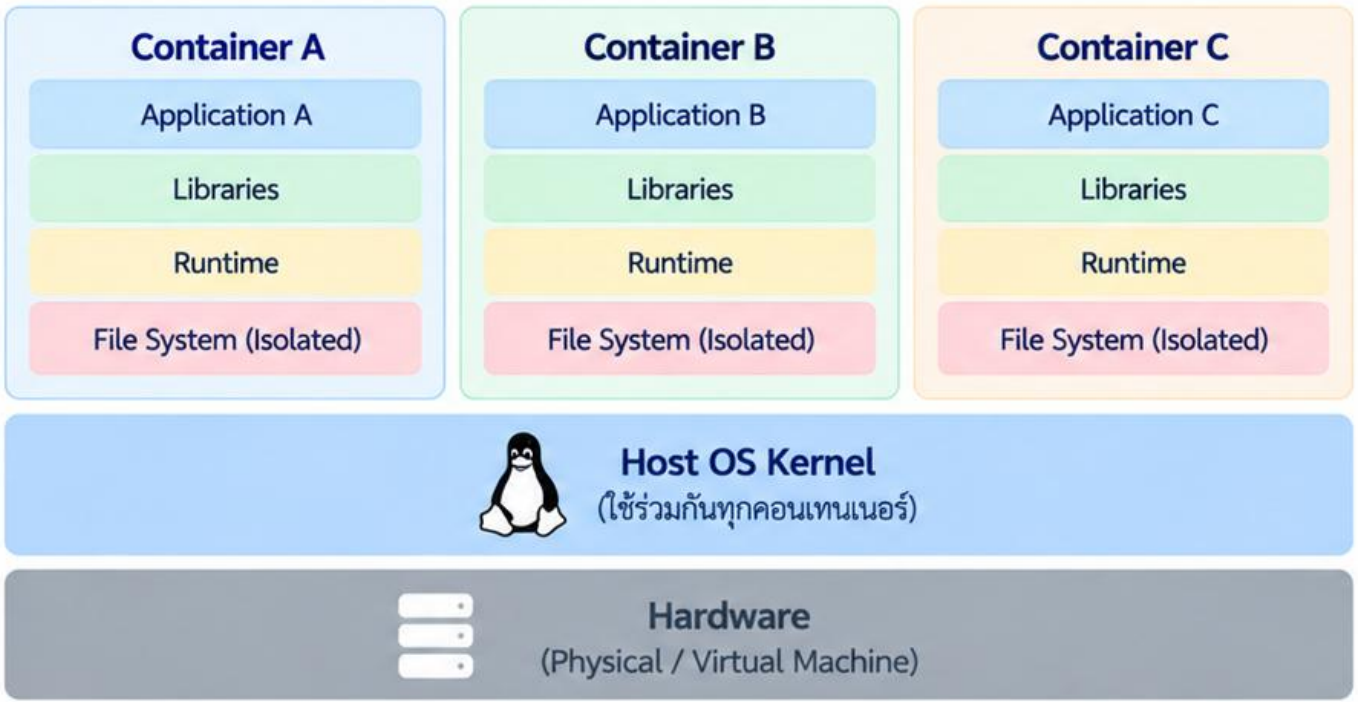
แนวคิดสำคัญ

“แพ็กแอปพลิเคชันพร้อมสภาพแวดล้อม → รันได้เหมือนกันทุกที่”

**Build once,
Run consistently.**

การแยกสภาพแวดล้อมของคอนเทนเนอร์ (Container Isolation)

คอนเทนเนอร์แต่ละตัวมีพื้นที่ใช้งานแยกจากกัน (isolated) แต่ใช้ Kernel ของระบบปฏิบัติการร่วมกัน ทำให้มีน้ำหนักเบาและเริ่มใช้งานได้รวดเร็ว



แยกกระบวนการ (Process Isolation)
แต่ละคอนเทนเนอร์เห็นเฉพาะกระบวนการของตัวเอง

แยกระบบไฟล์ (Filesystem Isolation)
แต่ละคอนเทนเนอร์มีไฟล์ระบบของตัวเอง

แยกเครือข่าย (Network Isolation)
แต่ละคอนเทนเนอร์มี Network Interface ของตัวเอง สื่อสารกันได้ตามที่กำหนด

ใช้ Kernel ร่วมกัน
ทุกคอนเทนเนอร์ใช้ Kernel ของ Host OS เดียวกัน จึงมีขนาดเล็กและเริ่มใช้งานได้เร็ว

ข้อดีคืออะไร?

แยกสภาพแวดล้อมได้เหมือนมีหลายเครื่อง แต่ไม่ต้องลงระบบปฏิบัติการหลายชุด จึงประหยัดทรัพยากรและจัดการได้ง่าย

Docker คืออะไร?

Docker คือแพลตฟอร์มที่ช่วยให้เราสร้าง แพ็ก จัดการ และรันแอปพลิเคชันในรูปแบบคอนเทนเนอร์ ได้อย่างง่ายดาย และทำงานได้เหมือนกันทุกสภาพแวดล้อม



สิ่งที่ Docker ช่วยให้เราทำได้

สร้าง (Build)
สร้างอิมเมจของแอปพลิเคชัน

แพ็ก (Package)
รวมทุกสิ่งที่จำเป็นไว้ในอิมเมจ

รัน (Run)
รันเป็นคอนเทนเนอร์ได้ทันที

จัดการ (Manage)
จัดการคอนเทนเนอร์ อิมเมจ และเครือข่าย

องค์ประกอบสำคัญของ Docker

Dockerfile	ไฟล์สำหรับกำหนดขั้นตอนการสร้างอิมเมจ
Image	แม่แบบของแอปพลิเคชันที่มีทุกสิ่งที่ต้องการ
Container	อินสแตนซ์ที่รันจากอิมเมจ
Docker Hub	ที่เก็บอิมเมจสาธารณะ (เช่น hub.docker.com)

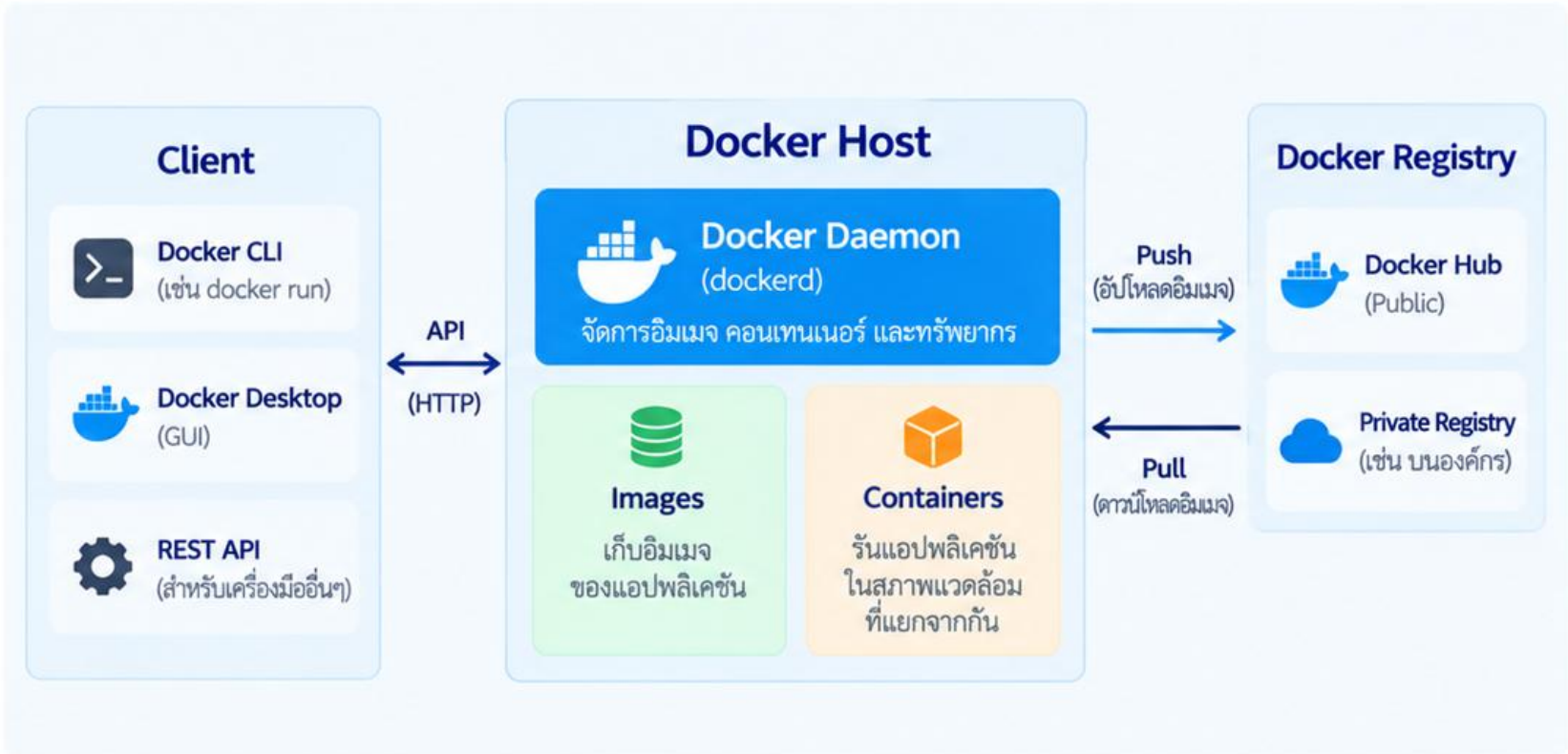
แนวคิดสำคัญ

Docker ทำให้การพัฒนา ทดสอบ และนำแอปพลิเคชันไปใช้งาน เป็นเรื่องง่าย รวดเร็ว และสอดคล้องกันในทุกสภาพแวดล้อม

Build, Ship, Run Anywhere

สถาปัตยกรรมของ Docker (Docker Architecture)

Docker ประกอบด้วยส่วนต่างๆ ที่ทำงานร่วมกัน เพื่อสร้าง จัดการ และรันคอนเทนเนอร์



หน้าที่ของแต่ละส่วน

Docker Client
เครื่องมือที่ใช้ส่งคำสั่งไปยัง Docker Daemon

Docker Daemon (dockerd)
ตัวหลักในการจัดการอิมเมจ คอนเทนเนอร์
เครือข่าย และทรัพยากร

Docker Images
แม่แบบของแอปพลิเคชัน (แบบอ่านอย่างเดียว)

Docker Containers
อินสแตนซ์ที่รันจากอิมเมจ

Docker Registry
แหล่งเก็บและแจกจ่ายอิมเมจ
เช่น Docker Hub หรือ Private Registry

สรุป

Docker ใช้สถาปัตยกรรมแบบ Client–Server โดยมี Docker Damon เป็นตัวกลางในการจัดการอิมเมจและคอนเทนเนอร์
และสามารถดึง/ส่งอิมเมจจาก Registry ได้

ขั้นตอนการใช้งาน Docker (Docker Workflow)

จากโค้ดของเรา ไปสู่คอนเทนเนอร์ที่รันได้ ภายในไม่กี่คำสั่ง

1 เขียน Dockerfile

กำหนดวิธีการสร้างอิมเมจ




```
FROM python:3.11
WORKDIR /app
COPY . .
RUN pip install -r
    requirements.txt
CMD ["python", "app.py"]
```

2 สร้างอิมเมจ

Build จาก Dockerfile



```
docker build -t
myapp:1.0 .
```

3 ตรวจสอบอิมเมจ

ดูรายการอิมเมจที่มี



```
docker images
```

4 รันคอนเทนเนอร์

สร้างและรันจากอิมเมจ



```
docker run -d -p
8080:80 --name myapp
myapp:1.0
```

5 ใช้งานแอปพลิเคชัน

เข้าถึงผ่านเบราว์เซอร์



แอปพลิเคชันทำงานอยู่
ในคอนเทนเนอร์



สรุป

Docker ช่วยให้เราสร้าง แพ็ก และรันแอปพลิเคชันได้อย่างรวดเร็วและสม่ำเสมอ ด้วยคำสั่งเพียงไม่กี่ขั้นตอน

ข้อดีของการใช้ Docker

- ทำงานเหมือนกันทุกสภาพแวดล้อม
- ติดตั้งและใช้งานได้รวดเร็ว
- ง่ายต่อการแชร์และนำไปใช้งานต่อ

Docker Image vs Docker Container

Image คือแม่แบบ (template) ส่วน Container คืออินสแตนซ์ที่กำลังทำงานจาก Image



Docker Image
แม่แบบของแอปพลิเคชัน
(อ่านอย่างเดียว)

`docker run` →



Docker Container
อินสแตนซ์ที่กำลังทำงาน
(อ่าน-เขียนได้)

- รวมแอปพลิเคชัน
- ไลบรารีและการตั้งค่า
- เป็นไฟล์แบบเลเยอร์
- นำกลับมาใช้ซ้ำได้

- รันจาก Image
- มีสภาพแวดล้อมของตัวเอง
- มีไฟล์ระบบที่เปลี่ยนแปลงได้
- เริ่ม/หยุด/ลบ ได้ง่าย

รายการ	Docker Image	Docker Container
ความหมาย	แม่แบบของแอปพลิเคชัน	อินสแตนซ์ที่กำลังทำงาน
สถานะ	ไฟล์ (ไม่ทำงาน)	กำลังทำงาน
การแก้ไข	ส่วนใหญ่เป็นแบบอ่านอย่างเดียว	อ่าน-เขียนได้ (มีการเปลี่ยนแปลงได้)
อายุการใช้งาน	คงอยู่จนกว่าลบ	ถูกสร้างและลบได้ตลอดเวลา
ขนาด	มักมีขนาดเล็ก (ใช้เลเยอร์ร่วมกัน)	เพิ่มเลเยอร์สำหรับการใช้งาน

คำสั่งที่เกี่ยวข้อง	`docker build`
	`docker pull`
		`docker run`
		`docker ps`
		`docker stop`



สรุป

Image เปรียบเหมือนพิมพ์เขียวของแอปพลิเคชัน ส่วน Container คือสิ่งที่ถูกสร้างขึ้นจาก Image เพื่อรันใช้งานจริง

Build once,
Run anywhere.

Dockerfile เบื้องต้น

Dockerfile คือไฟล์ข้อความที่บอกขั้นตอนในการสร้าง Image ของเรา

ตัวอย่าง Dockerfile (Python Web App)

```
1  FROM python:3.11-slim
2  WORKDIR /app
3  COPY requirements.txt .
4  RUN pip install -r requirements.txt
5
6  COPY app.py .
7  EXPOSE 8080
8  CMD ["python", "app.py"]
9
10
```

อธิบายแต่ละคำสั่ง

FROM	กำหนด Image ฐาน (Base Image)
WORKDIR	กำหนดไดเรกทอรีสำหรับทำงานในคอนเทนเนอร์
COPY	คัดลอกไฟล์จากเครื่องของเราเข้าไปใน Image
RUN	รันคำสั่งระหว่างการสร้าง Image (เช่น ติดตั้งไลบรารี)
EXPOSE	ระบุพอร์ตที่แอปใช้ (เป็นเอกสารไม่ใช่การเปิดพอร์ต)
CMD	คำสั่งเริ่มต้นเมื่อรันคอนเทนเนอร์

คำสั่งที่ใช้สร้างและรัน

1 สร้าง Image จาก Dockerfile

```
$ docker build -t myapp:1.0 .
```

2 รันคอนเทนเนอร์จาก Image

```
$ docker run -d -p 8080:8080 myapp:1.0
```

สรุป

Dockerfile ช่วยให้เราสร้าง Image ได้อย่างอัตโนมัติ ทำซ้ำได้ และแชร์ให้ผู้อื่นใช้งานได้ง่าย โดยระบุขั้นตอนการติดตั้งและตั้งค่าแอปพลิเคชันทั้งหมด

แนวทางปฏิบัติที่ดี

- ใช้ Image ที่เชื่อถือได้ (เช่น official image)
- ลดขนาด Image โดยใช้ base image ขนาดเล็ก
- ไม่ใส่ข้อมูลลับใน Dockerfile

คำสั่งสำคัญใน Dockerfile

เรียนรู้คำสั่งพื้นฐานที่ใช้บ่อยในการสร้าง Image ของแอปพลิเคชัน

คำสั่ง	ความหมาย	ตัวอย่าง
FROM	กำหนด Image ฐาน (Base Image)	`FROM python:3.11-slim`
WORKDIR	กำหนดไดเรกทอรีในการทำงาน	`WORKDIR /app`
COPY	คัดลอกไฟล์จากเครื่องเราเข้าไปใน Image	`COPY . /app`
RUN	รันคำสั่งระหว่างการสร้าง Image เช่น ติดตั้งแพ็กเกจ	`RUN pip install -r requirements.txt`
EXPOSE	ระบุพอร์ตที่แอปใช้ (เป็นเอกสาร ไม่ได้เปิดพอร์ตจริง)	`EXPOSE 8080`
CMD	คำสั่งเริ่มต้นเมื่อรันคอนเทนเนอร์	`CMD ["python", "app.py"]`

ตัวอย่าง Dockerfile (สรุป)

```
1  FROM python:3.11-slim
2  WORKDIR /app
3  COPY requirements.txt .
4  RUN pip install -r requirements.txt
5  COPY app.py .
6  EXPOSE 8080
7  CMD ["python", "app.py"]
```

ข้อควรระวัง

- เลือก Base Image ที่เหมาะสมและเชื่อถือได้
- หลีกเลี่ยงการใส่ข้อมูลลับใน Dockerfile (เช่น รหัสผ่าน)
- ใช้ .dockerignore เพื่อไม่ให้คัดลอกไฟล์ที่ไม่จำเป็น
- ลดขนาด Image ให้เล็กที่สุด (เช่น ใช้ slim image)

สรุป

Dockerfile ประกอบด้วยคำสั่งที่ละบรรทัด เพื่อกำหนดขั้นตอนการสร้าง Image เมื่อ build แล้วจะได้ Image ที่พร้อมใช้งานบนทุกสภาพแวดล้อม

“Infrastructure as Code”

แอปพลิเคชันของคุณ สามารถถูกสร้างซ้ำได้ทุกที่ ด้วย Dockerfile เพียงไฟล์เดียว

การจัดการคอนเทนเนอร์ด้วย Docker CLI

คำสั่งพื้นฐานที่ใช้บ่อยในการทำงานกับคอนเทนเนอร์

1 รันคอนเทนเนอร์

สร้างและรันคอนเทนเนอร์จาก Image

```
$ docker run -d -p 8080:80 \
  -name myapp nginx
```

`-d` รันแบบเบื้องหลัง (detached)
`-p` แมปพอร์ต, `--name` ตั้งชื่อคอนเทนเนอร์

2 ดูรายการคอนเทนเนอร์

แสดงคอนเทนเนอร์ที่กำลังทำงาน

```
$ docker ps
```

ใช้ `docker ps -a` เพื่อดูคอนเทนเนอร์ทั้งหมด (รวมที่หยุดแล้ว)

3 หยุดคอนเทนเนอร์

หยุดการทำงานของคอนเทนเนอร์

```
$ docker stop myapp
```

สามารถใช้ container ID แทนชื่อคอนเทนเนอร์ได้

4 ลบคอนเทนเนอร์

ลบคอนเทนเนอร์ที่ไม่ต้องการ

```
$ docker rm myapp
```

หากคอนเทนเนอร์ยังทำงานอยู่ ให้หยุดก่อน (docker stop)

5 ดูบันทึกการทำงาน

แสดง log ของคอนเทนเนอร์

```
$ docker logs myapp
```

ใช้ `-f` เพื่อติดตาม log แบบเรียลไทม์
เช่น `docker logs -f myapp`

6 เข้าใช้งานภายในคอนเทนเนอร์

เปิด shell เพื่อทำงานภายในคอนเทนเนอร์

```
$ docker exec -it myapp /bin/bash
```

`-it` เปิด interactive terminal
`/bin/bash` คือ shell ภายในคอนเทนเนอร์

7 ดึงอิมเมจล่าสุด

ดาวน์โหลด Image จาก Docker Hub

```
$ docker pull nginx:latest
```

ระบุ tag ของ Image ที่ต้องการ
เช่น `nginx:1.25, python:3.11`

สรุป

Docker CLI มีคำสั่งที่ไม่ซับซ้อน แต่ทรงพลัง ช่วยให้เราจัดการคอนเทนเนอร์ได้อย่างรวดเร็ว ตั้งแต่การรัน ตรวจสอบ หยุด ลบ รวมถึงการดู log และเข้าใช้งานภายใน

เคล็ดลับเพิ่มเติม

- ใช้ `docker --help` เพื่อดูคำสั่งเพิ่มเติม
- ใช้ชื่อที่สื่อความหมายให้คอนเทนเนอร์
- สามารถใช้ container ID แทนชื่อได้ (บางคำสั่ง)

การสื่อสารระหว่างคอนเทนเนอร์ (Networking)

ให้คอนเทนเนอร์เชื่อมต่อกัน และเข้าถึงจากภายนอกได้อย่างปลอดภัย

1 โครงข่ายเริ่มต้น (Default Bridge)

- คอนเทนเนอร์เชื่อมต่อกันได้อัตโนมัติ บนเครือข่าย bridge
- ใช้ชื่อคอนเทนเนอร์ในการสื่อสาร
- เหมาะสำหรับการพัฒนาและทดสอบ



2 การแมปพอร์ต (Port Mapping)

- เปิดพอร์ตจากโฮสต์ให้เข้าถึงคอนเทนเนอร์
- รูปแบบ: -p <host_port>:<container_port>

```
$ docker run -d -p 8080:80 --name web nginx
```



3 การสร้างเครือข่ายแบบกำหนดเอง

- ใช้ docker network create เพื่อสร้างเครือข่ายแยกเฉพาะแอปพลิเคชัน
- ปลอดภัยและจัดการได้ง่าย
- คอนเทนเนอร์ในเครือข่ายเดียวกันสื่อสารกันด้วยชื่อได้

```
$ docker network create mynet
$ docker run -d --network mynet --name db mysql
$ docker run -d --network mynet --name app python
```



สรุป

Docker มีระบบเครือข่ายที่ช่วยให้คอนเทนเนอร์สื่อสารกันได้ง่าย และสามารถเปิดพอร์ตเพื่อให้บริการจากภายนอกได้อย่างปลอดภัย

เคล็ดลับเพิ่มเติม

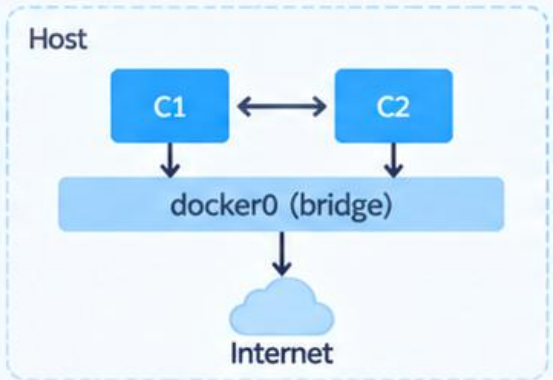
- ใช้ชื่อคอนเทนเนอร์แทน IP (เช่น http://app:8000)
- เปิดเฉพาะพอร์ตที่จำเป็น
- ใช้ custom network สำหรับโปรเจกต์ที่มีหลายคอนเทนเนอร์

ประเภทของ Docker Network

เลือกใช้โหมดเครือข่ายให้เหมาะสมกับการใช้งาน

1 Bridge (ค่าเริ่มต้น)

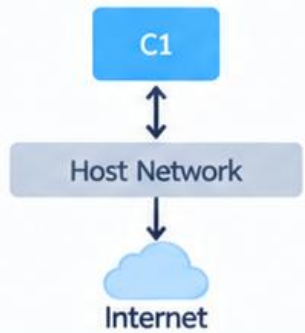
เครือข่ายภายในเครื่อง (Host) คอนเทนเนอร์สื่อสารกันได้ และออกอินเทอร์เน็ตได้




```
# ใช้ค่าเริ่มต้น (bridge)
docker run -d --name web nginx
```

2 Host

ใช้เครือข่ายเดียวกับ Host คอนเทนเนอร์เข้าถึงพอร์ตของเครื่องโดยตรง (ไม่มี Network isolation)




```
docker run --network host
nginx
```

3 None

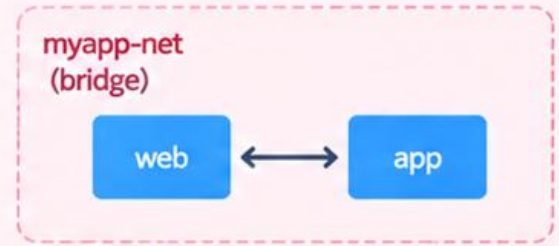
ไม่มีการเชื่อมต่อเครือข่าย คอนเทนเนอร์ไม่สามารถสื่อสารกับภายนอกได้




```
docker run --network none
nginx
```

4 Custom Network

สร้างเครือข่ายเองเพื่อแยกแอปพลิเคชัน สามารถกำหนด subnet, ชื่อเครือข่าย และควบคุมการสื่อสารได้




```
# สร้างเครือข่าย
docker network create myapp-net

# รันคอนเทนเนอร์ในเครือข่ายที่สร้าง
docker run -d --network myapp-net
--name web nginx
```

สรุป

Docker มีโหมดเครือข่ายหลายแบบ แต่ละแบบเหมาะกับงานที่แตกต่างกัน
ควรเลือกใช้ให้เหมาะสมกับความต้องการด้านความปลอดภัยและการสื่อสารของแอปพลิเคชัน

แนวทางการเลือกใช้

- งานทั่วไป ใช้ bridge (ค่าเริ่มต้น)
- ต้องการประสิทธิภาพสูง ใช้ host
- ต้องการแยกแอปพลิเคชัน ใช้ custom network

Docker Volume และการจัดการข้อมูลถาวร

เก็บข้อมูลนอกคอนเทนเนอร์ เพื่อให้ข้อมูลไม่หายเมื่อคอนเทนเนอร์ถูกลบ

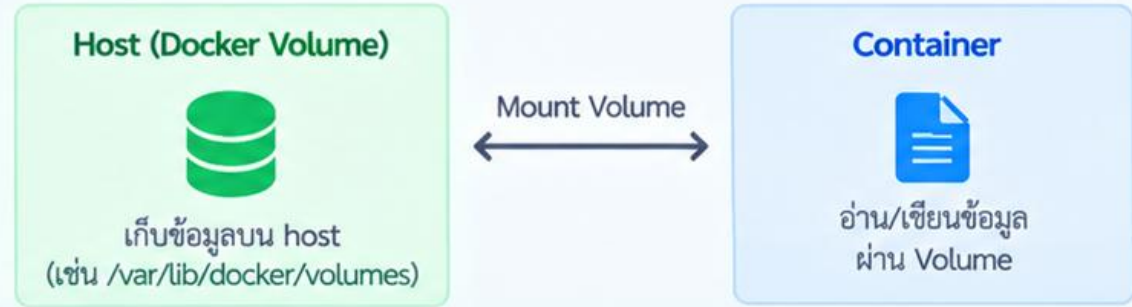
ปัญหาของคอนเทนเนอร์

ข้อมูลที่เขียนลงในคอนเทนเนอร์จะอยู่ในชั้น writable layer
ซึ่งจะหายไปเมื่อคอนเทนเนอร์ถูกลบ



Docker Volume

เก็บข้อมูลไว้บนโฮสต์ (นอกคอนเทนเนอร์) สามารถใช้ซ้ำได้ แม้ลบคอนเทนเนอร์



ตัวอย่างการใช้งาน

สร้างและใช้งาน Volume

```
# สร้าง volume
docker volume create mydata

# ใช้งาน volume กับคอนเทนเนอร์
docker run -d --name web \\
  -p 8080:80 \
  -v mydata:/usr/share/nginx/html \
  nginx
```

ตรวจสอบและลบ Volume

```
# ดูรายการ volume
docker volume ls

# ตรวจสอบรายละเอียด
docker volume inspect mydata

# ลบ volume (เมื่อไม่ใช้งานแล้ว)
docker volume rm mydata
```

กรณีใช้งานที่พบบ่อย

- **Database**
  เก็บข้อมูลฐานข้อมูล (เช่น MySQL, PostgreSQL)
- **Application Data**
  เก็บไฟล์ที่อัปโหลดหรือไฟล์ที่แอปพลิเคชันสร้าง
- **Configuration**
  เก็บไฟล์ตั้งค่าที่ต้องการใช้งานต่อเนื่อง
- **Backup / Data Persistence**
  รักษาข้อมูลเมื่อมีการอัปเดตหรือลบคอนเทนเนอร์

Docker Registry และ Docker Hub

แหล่งเก็บและแจกจ่าย Image สำหรับใช้งานร่วมกัน

การทำงานของ Docker Registry

ดึง Image จาก Registry นำมาสร้างและรันเป็น Container



Docker Hub
(Public Registry)
มี Image ให้ใช้งานมากมาย

Image
เช่น nginx:latest
python:3.11

Container
รันจาก Image
เป็นแอปพลิเคชันที่ใช้งานได้

ประเภทของ Registry

Public Registry

- ทุกคนสามารถเข้าถึงได้
- เช่น Docker Hub, GitHub Container Registry

Private Registry

- จำกัดการเข้าถึงเฉพาะองค์กร
- เช่น DockerHub (Private), AWS ECR, Azure ACR, Google Artifact Registry

Docker Hub

Registry สาธารณะที่ได้รับความนิยมมากที่สุด

- มี Image อย่างเป็นทางการ (Official Images)
- ใช้งานฟรี (มีข้อจำกัดสำหรับอัตราการใช้งาน)
- สามารถค้นหาและดาวน์โหลด Image ได้ง่าย
- รองรับทั้ง Public และ Private Repository
- สมัครใช้งานได้ที่ https://hub.docker.com

docker hub

https://hub.docker.com

ตัวอย่างคำสั่ง

```
# ค้นหา Image
docker search nginx

# ดาวน์โหลด Image
docker pull nginx:latest

# ดู Image ที่มีในเครื่อง
docker images
```

สรุป

Docker Registry เป็นแหล่งเก็บ Image ส่วน Docker Hub คือ Registry สาธารณะที่นิยม ช่วยให้เราสามารถแชร์และใช้งาน Image ได้อย่างสะดวกและรวดเร็ว

เคล็ดลับเพิ่มเติม

- ใช้ tag ของ Image เช่น nginx:1.25, python:3.11
- ตรวจสอบความน่าเชื่อถือของ Image (Official Image)
- ในงานจริง อาจใช้ Private Registry เพื่อความปลอดภัย

Container Lifecycle

วงจรชีวิตของคอนเทนเนอร์ ตั้งแต่สร้าง รัน หยุด จนถึงลบ

1 Created
สร้างคอนเทนเนอร์
(ยังไม่ได้รัน)

→

2 Running
คอนเทนเนอร์กำลังทำงาน
(ให้บริการ/รันแอปพลิเคชัน)

→

3 Stopped
คอนเทนเนอร์หยุดทำงาน
(ยังคงอยู่แต่ไม่รัน)

→

4 Removed
ลบคอนเทนเนอร์
(ไม่อยู่ในระบบอีกต่อไป)

1 สร้างคอนเทนเนอร์

```
$ docker create nginx
$ docker run -d --name web1 nginx
```

- `create` : สร้างแต่ยังไม่รัน
- `run` : สร้างและรันทันที (นิยมใช้มากที่สุด)

2 จัดการการทำงาน

```
$ docker start web1
$ docker stop web1
$ docker restart web1
```

- `start` : เริ่มรันคอนเทนเนอร์ที่หยุดอยู่
- `stop` : หยุดการทำงาน
- `restart` : หยุดแล้วเริ่มใหม่

3 ตรวจสอบสถานะ

```
$ docker ps          # ดูที่กำลังรัน
$ docker ps -a       # ดูทั้งหมด
$ docker inspect web1
```

- `ps` : ดูคอนเทนเนอร์ที่กำลังรัน
- `ps -a` : ดูทุกคอนเทนเนอร์
- `inspect` : ดูรายละเอียด (IP, config, ฯลฯ)

4 ลบคอนเทนเนอร์

```
$ docker rm web1
$ docker rm -f web1
```

- `rm` : ลบคอนเทนเนอร์ (ต้องหยุดก่อน)
- `rm -f` : บังคับลบ (แม้กำลังรัน)

สรุป

Docker มีคำสั่งสำหรับจัดการคอนเทนเนอร์ตลอดทั้งวงจรชีวิต ตั้งแต่สร้าง รัน หยุด ตรวจสอบ ไปจนถึงลบ ช่วยให้เราจัดการแอปพลิเคชันได้อย่างยืดหยุ่น

เคล็ดลับเพิ่มเติม

- ใช้ docker ps -a เพื่อตรวจสอบคอนเทนเนอร์ที่หยุดอยู่
- ใช้ docker logs <name> เพื่อดู log ของคอนเทนเนอร์
- ใช้ docker rm $(docker ps -aq) เพื่อลบทั้งหมด (ระวัง!)

การจัดการทรัพยากรของคอนเทนเนอร์ (Resource Management)

กำหนดการใช้ CPU และหน่วยความจำ เพื่อให้ระบบทำงานได้อย่างเสถียรและใช้ทรัพยากรอย่างเหมาะสม

1 จำกัดการใช้ CPU

กำหนดจำนวน CPU (หรือสัดส่วน) ที่คอนเทนเนอร์สามารถใช้งานได้

```
$ docker run -d --name web \
  --cpus=0.5 nginx
```

- `--cpus=0.5` ใช้ได้ไม่เกิน 50% ของ 1 CPU
- `--cpus=2` ใช้ได้สูงสุด 2 CPU

2 จำกัดการใช้หน่วยความจำ (Memory)

กำหนดขนาดหน่วยความจำที่คอนเทนเนอร์สามารถใช้งานได้

```
$ docker run -d --name web \
   --memory=512m nginx
```

- `--memory=512m` จำกัดที่ 512 MB
- `--memory=1g` จำกัดที่ 1 GB
- `--memory=2g` จำกัดที่ 2 GB

3 ตัวอย่าง: จำกัดทั้ง CPU และ Memory

```
$ docker run -d --name app \
  --cpus=1 \
  --memory=1g \
  -p 8080:80 nginx
```

- จำกัด CPU 1 vCPU
- จำกัดหน่วยความจำ 1 GB
- เปิดพอร์ต 8080 → 80
- รันแอปพลิเคชัน Nginx

ความสัมพันธ์กับการจัดสรรทรัพยากรของ VM (Host)

ตัวอย่าง: VM ขนาด 4 vCPU / 8 GB RAM



สามารถแบ่งทรัพยากรให้หลายคอนเทนเนอร์บน VM เดียวกันได้

สิ่งที่ควรระวัง

หากไม่กำหนด limit
คอนเทนเนอร์อาจใช้ทรัพยากรมากเกินไปและกระทบคอนเทนเนอร์อื่น

หน่วยความจำไม่พอ (OOM)
อาจถูกระบบสั่งหยุด (Out of Memory)

ควรทดสอบโหลดจริง
เพื่อเลือกขนาดทรัพยากรที่เหมาะสม

แนวทางปฏิบัติที่ดี

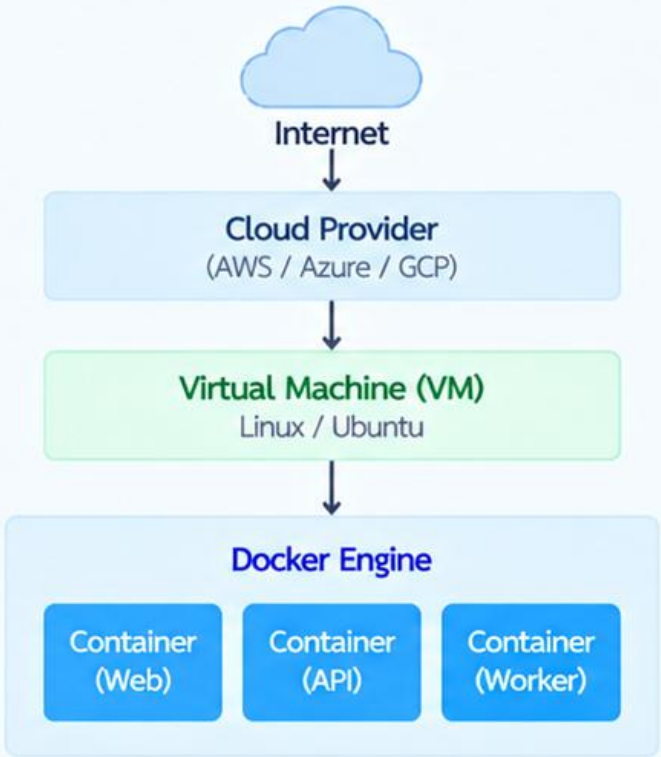
- กำหนด CPU และ Memory limit สำหรับทุกคอนเทนเนอร์
- ติดตามการใช้งานด้วยคำสั่ง เช่น `docker stats`
- เลือกขนาดทรัพยากรให้เหมาะกับงานและเผื่อการขยายในอนาคต
- หากเป็นงาน production ควรมีการ monitoring อย่างต่อเนื่อง

Containers on Cloud

การใช้งาน Docker บนสภาพแวดล้อมคลาวด์ เพื่อความยืดหยุ่นและขยายได้ง่าย

แนวคิดการทำงาน

สามารถรัน Docker บนเครื่องเสมือน (VM) หรือใช้บริการ Container ที่ผู้ให้บริการคลาวด์จัดเตรียมให้



รูปแบบการใช้งานบนคลาวด์

1 Docker บน VM

ติดตั้ง Docker Engine บน VM ที่สร้างจากบริการ IaaS

- Amazon EC2
- Azure Virtual Machines
- Google Compute Engine

- ควบคุมสภาพแวดล้อมได้เต็มที่
- เหมาะสำหรับงานทั่วไป
- ติดตั้ง Docker ได้เอง
- จัดการ VM และ Container เอง

2 Managed Container Service

ใช้บริการที่ผู้ให้บริการจัดเตรียมและดูแลให้ (เช่น Kubernetes)

- Amazon EKS (Elastic Kubernetes Service)
- Azure AKS (Azure Kubernetes Service)
- Google GKE (Google Kubernetes Engine)

- ไม่ต้องดูแลเครื่อง VM เอง
- ขยายขนาดได้อัตโนมัติ
- เหมาะสำหรับงานระดับ production
- มีเครื่องมือเสริม เช่น load balancing, monitoring, auto scaling

เปรียบเทียบแนวทาง

หัวข้อ	Docker บน VM	Managed Container Service
การดูแล	ดูแลเองทั้งหมด	ผู้ให้บริการดูแลให้
ความยืดหยุ่น	สูง (ปรับแต่งได้เอง)	สูง (มีฟีเจอร์พร้อมใช้)
ความซับซ้อน	ปานกลาง	ต่ำ
เหมาะสำหรับ	เรียนรู้ / งานขนาดเล็กถึงกลาง	งานขนาดใหญ่ระดับ production
ค่าใช้จ่าย	ตามขนาด VM	คิดตามการใช้งาน (Pay as you go)

สรุป

Docker สามารถใช้งานบนคลาวด์ได้ทั้งแบบติดตั้งบน VM หรือใช้บริการ Container แบบจัดการ (Managed Service) ขึ้นอยู่กับความต้องการ ความสะดวก และขนาดของงาน

ข้อควรพิจารณา

- เลือกบริการให้เหมาะกับความต้องการ
- พิจารณาค่าใช้จ่ายในระยะยาว
- คำนึงถึงความปลอดภัยและการจัดการสิทธิ์
- เตรียมการขยายระบบในอนาคต

When Should We Use Containers?

เหมาะกับงานแบบไหน และเมื่อไรไม่ควรใช้

งานที่เหมาะสม (Suitable Use Cases)

Containers เหมาะกับงานที่ต้องการความยืดหยุ่น รวดเร็ว และขยายได้ง่าย

Web Application
เว็บไซต์, เว็บแอปพลิเคชัน

API / Microservices
บริการย่อย ขยายแยกกันได้

Background Worker
งานประมวลผลเบื้องหลัง
เช่น queue, batch

CI/CD
ใช้ใน pipeline
เพื่อ build/test/deploy

Development and Testing
สร้างสภาพแวดล้อมได้เร็ว
เหมือน production

Data Processing
งานประมวลผลข้อมูล
แบบชั่วคราว

งานที่ควรพิจารณาอย่างรอบคอบ (Use with Caution)

สามารถใช้ Container ได้ แต่ต้องวางแผนเพิ่มเติม

Database
ข้อมูลมีสถานะ (stateful)
ต้องจัดการ storage,
backup, และการกู้คืน

File Storage
ระบบไฟล์ที่ต้องเก็บข้อมูลถาวร
ควรใช้ Volume หรือ
บริการจัดเก็บข้อมูลภายนอก

Legacy Application
แอปพลิเคชันเดิมที่อาจไม่รองรับ
การทำงานบน Container

High Compliance
งานที่มีข้อกำหนดความปลอดภัย
หรือกฎหมายเฉพาะ
อาจต้องมีการออกแบบเพิ่มเติม

งานที่ไม่เหมาะสม (Not Suitable)

อาจไม่คุ้มค่า หรือมีทางเลือกที่ดีกว่า

Monolithic Application ขนาดใหญ่
แอปพลิเคชันขนาดใหญ่มาก
ที่ไม่สามารถแยกส่วนได้ง่าย

งานที่ต้องการประสิทธิภาพระดับสูงมาก
เช่น งานที่ต้องเข้าถึงฮาร์ดแวร์โดยตรง
(High-performance computing, GPU
แบบเฉพาะเจาะจง)

ข้อแนะนำ

ไม่มีเทคโนโลยีใดเหมาะกับทุกงาน
ควรพิจารณาความต้องการของระบบ
ต้นทุน ทักษะของทีม และแผนการขยาย
ในอนาคต

สรุป

Containers เหมาะกับงานสมัยใหม่ที่ต้องการความรวดเร็ว ยืดหยุ่น และขยายได้ง่าย
แต่สำหรับงานที่มีข้อมูลถาวร ข้อกำหนดเฉพาะ หรือความต้องการประสิทธิภาพสูง
ควรประเมินอย่างรอบคอบและอาจใช้เทคโนโลยีอื่นร่วมด้วย

คำถามที่ควรถามก่อนเลือกใช้

- งานของเรามีสถานะของข้อมูล (stateful) หรือไม่?
- ต้องการขยายระบบบ่อยแค่ไหน?
- ทีมมีความคุ้นเคยกับ Docker หรือไม่?
- มีข้อกำหนดด้านความปลอดภัย/กฎหมายเฉพาะหรือไม่?
- ต้นทุนและการดูแลระยะยาวเป็นอย่างไร?

Summary: Container & Docker Fundamentals

จากโค้ดสู่คอนเทนเนอร์ พร้อมใช้งานบนคลาวด์

1 Dockerfile



ไฟล์ที่กำหนดขั้นตอนการสร้าง Image

```
FROM nginx:alpine
COPY . /usr/share/nginx/html
```

2 Image



ผลลัพธ์จากการ build เป็นเทมเพลตสำหรับสร้าง Container

```
myweb:v1
```

3 Registry



แหล่งเก็บและแจกจ่าย Image เช่น Docker Hub หรือ Private Registry

```
docker push
docker pull
```

4 Container



รันจาก Image เป็นแอปพลิเคชันที่ใช้งานได้

```
docker run
```

5 Network



เชื่อมต่อ Container กับภายนอกหรือ Container อื่น

```
-p 8080:80
--network ...
```

6 Volume



จัดเก็บข้อมูลแบบถาวร แยกจาก Container

```
-p d0ta:/data
```



Key Takeaways

1. Container คือหน่วยรันแอปพลิเคชันที่เบาและพกพาได้
2. Docker ช่วยให้สร้าง จัดการ และเผยแพร่ Container ได้ง่าย
3. ใช้ Registry เพื่อแชร์ Image
4. ใช้ Network และ Volume เพื่อสร้างระบบที่สมบูรณ์
5. สามารถใช้งานบน VM หรือบริการคลาวด์ได้



Benefits

- พัฒนาและทดสอบได้รวดเร็ว
- สภาพแวดล้อมเหมือนกันทุกที่
- ใช้ทรัพยากรได้อย่างมีประสิทธิภาพ
- เหมาะกับงานสมัยใหม่ (Cloud, CI/CD, Microservices)
- ขยายระบบได้ง่าย



Next Step

ในหัวข้อต่อไป เราจะได้ลงมือปฏิบัติจริง (Workshop) เพื่อใช้คำสั่ง Docker และสร้างเว็บแอปพลิเคชันของตัวเองด้วย Dockerfile

Free Docker Playground for Workshop

ฝึกใช้ Docker ผ่าน Browser โดยไม่ต้องติดตั้งโปรแกรม

KILLERCODA

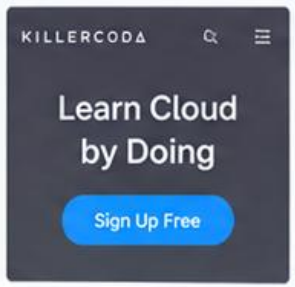
Learn by Doing

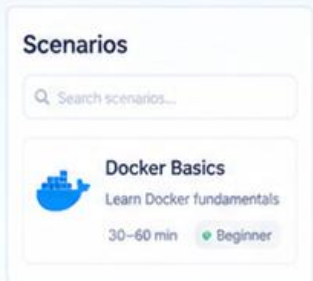
Killercoda คือแพลตฟอร์มสำหรับฝึกใช้งานผ่าน Browser ให้คุณมี Linux environment พร้อม Docker โดยไม่ต้องติดตั้งอะไรบนเครื่อง

- ใช้งานฟรี (Free tier)
- มี Docker CLI พร้อมใช้งาน
- เหมาะสำหรับการทำ Workshop ในห้องเรียน

วิธีเริ่มใช้งาน (ตัวอย่าง)

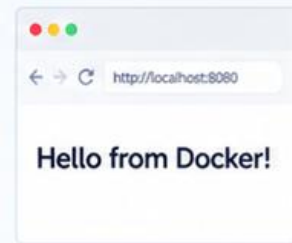
1. ไปที่เว็บไซต์ killercoda.com
2. สมัครบัญชี (ฟรี) และเลือก Scenario
3. เริ่ม Environment ผ่าน Browser
4. ใช้คำสั่ง Docker ได้ทันที
5. ทำ Workshop ตามโจทย์











สิ่งที่สามารถทำได้

- รันคำสั่ง Docker CLI ทุกคำสั่งที่เรียนในวันนี้
- สร้างและ build Dockerfile
- รัน Web application ใน Container
- ตรวจสอบ logs, inspect, stop/start container
- เหมาะสำหรับ Workshop 1 และ Workshop 2

ข้อควรรู้

- ฟรีสำหรับการใช้งานส่วนบุคคล (Free tier)
- แต่ละ session ใช้งานได้นานสูงสุดประมาณ 1 ชั่วโมง
- เมื่อหมดเวลา สามารถเริ่ม environment ใหม่ได้
- ไม่ต้องติดตั้งโปรแกรมใด ๆ บนเครื่อง
- ต้องมีบัญชีผู้ใช้งาน (Sign up)

เคล็ดลับ

- ใช้ Scenario ที่เกี่ยวกับ Docker โดยตรง
- สามารถแชร์ลิงก์ Scenario ให้นักศึกษาได้
- เหมาะสำหรับการเรียนในห้องเรียนและการบ้าน
- นอกจาก Docker ยังมี Linux, Kubernetes และเทคโนโลยีอื่น ๆ ให้ทดลองด้วย

Workshop 1: Docker CLI Fundamentals

ฝึกใช้งานคำสั่งพื้นฐานของ Docker กับ Image สำเร็จรูป

1 ตรวจสอบการติดตั้ง

ตรวจสอบว่า Docker พร้อมใช้งาน

```
$ docker --version
$ docker info
```

2 ดาวน์โหลด Image

ดึง Image จาก Docker Hub

```
$ docker pull nginx
$ docker images
```

3 สร้างและรัน Container

รันเว็บเซิร์ฟเวอร์ Nginx และเปิดพอร์ต

```
$ docker run -d \
  --name web1 \
  -p 8080:80 nginx
```

4 ตรวจสอบสถานะ

ดู Container ที่กำลังทำงาน

```
$ docker ps       # ดูที่กำลังรัน
$ docker ps -a    # ดูทั้งหมด
```

5 จัดการการทำงาน

หยุด เริ่ม และรีสตาร์ท Container

```
$ docker stop web1
$ docker start web1
$ docker restart web1
```

6 ดูรายละเอียดและ Log

ตรวจสอบข้อมูลและดู Log

```
$ docker logs web1
$ docker inspect web1
```

7 เข้าไปใน Container

เข้าใช้งานภายใน Container

```
$ docker exec -it web1 /bin/bash
# ภายใน Container
$ ls
$ cd /usr/share/nginx/html
$ cat index.html
```

8 ลบ Container และ Image

หยุดและลบ Container และ Image

```
$ docker stop web1
$ docker rm web1
$ docker images
$ docker rmi nginx
```

เป้าหมายของ Workshop

- เข้าใจวงจรการทำงานของ Docker: Registry → Image → Container → Management
- สามารถใช้งานคำสั่งพื้นฐานของ Docker ได้ด้วยตนเอง
- เตรียมความพร้อมสำหรับการสร้าง Image ของตนเองด้วย Dockerfile ใน Workshop ถัดไป

ลองทำดู

1. เปิดเว็บที่ http://localhost:8080 ในเบราว์เซอร์
2. แก้ไขไฟล์ index.html ภายใน Container แล้วดูผลลัพธ์

Workshop 2: Build a Simple Web Application with Docker

สร้างและรันเว็บแอปพลิเคชันด้วย Docker แบบง่าย ๆ

1 เตรียมไฟล์โปรเจกต์

สร้างไฟล์สำหรับเว็บแอป (เช่น HTML เล็ก ๆ)

```
myweb/
├── index.html
└── Dockerfile
```

index.html (ตัวอย่าง)

```
<!DOCTYPE html>
<html>
 <head><title>My Web</title></head>
 <body>
  <h1>Hello from Docker!</h1>
 </body>
</html>
```

2 เขียน Dockerfile

กำหนด Image และคำสั่งสำหรับรันเว็บเซิร์ฟเวอร์

Dockerfile (ตัวอย่าง)

```
FROM nginx:alpine
COPY index.html
 /usr/share/nginx/html/
EXPOSE 80
```

3 สร้าง Image

ใช้คำสั่ง docker build เพื่อสร้าง Image จาก Dockerfile

```
$ docker build -t
   myweb:1.0 .
```

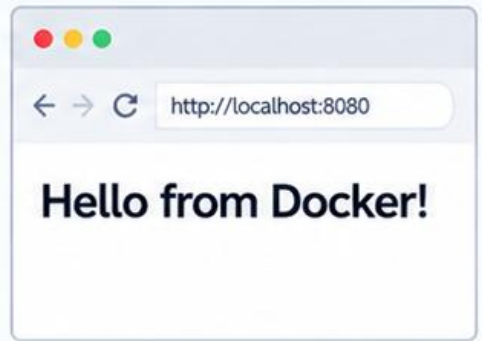
4 รัน Container

สร้างและรัน Container จาก Image

```
$ docker run -d \
  --name myweb \
  -p 8080:80 \
  myweb:1.0
```

5 ทดสอบการใช้งาน

เปิดเบราว์เซอร์ไปที่ http://localhost:8080



เป้าหมายของ Workshop

- เข้าใจขั้นตอนการสร้าง Docker Image และรัน Container
- สามารถปรับแต่งเว็บแอปของตนเองได้
- รู้จักการแมปพอร์ต (Port Mapping)
- เห็นว่าการใช้ Docker ช่วยให้การพัฒนาและทดสอบแอปพลิเคชันทำได้ง่ายขึ้น

ลองทำดูเพิ่มเติม

1. เปลี่ยนข้อความใน index.html แล้ว build image ใหม่
2. ลองใช้พอร์ตอื่น เช่น 8081
3. เพิ่มไฟล์ CSS หรือรูปภาพในเว็บเพจ



Q & A

Thank You

ขอบคุณสำหรับการมีส่วนร่วม
แล้วพบกันในหัวข้อถัดไปครับ